

SHL Recommendation Agent — Approach Summary

Problem framing - Provide deterministic, explainable recommendations of SHL assessments from an official catalog. Avoid hallucination; ensure schema compliance and stateless server behavior.

Architecture - FastAPI service exposing `/chat` (stateless) and `/health` endpoints. - Retriever: hybrid BM25 + FAISS embeddings with deterministic feature-based boosts. - Orchestrator: deterministic clarify/refuse/compare/recommend flow; LLM only for phrasing and light rerank.

Retrieval setup - BM25 index over catalog text + FAISS dense index over normalized embeddings. - Deterministic boosts for skill overlap, metadata completeness, specific assessment pages, and URL depth. - Calibration performed via `evaluation/calibration.py` grid-search; best balanced config selected.

Prompting & orchestration - Clarification questions generated deterministically when queries are ambiguous. - Off-topic and prompt-injection detection live in orchestrator before any LLM call.

Evaluation approach - Behavioral probes test clarify/refuse/compare/refinement behaviors. - Schema checks ensure responses match `ChatResponse` Pydantic model. - Recall proxy and retrieval diagnostics evaluate retrieval effectiveness.

What failed & iterations - Early reliance on LLM for clarification produced inconsistent behavior — moved to deterministic rules. - Initial crawler produced many category pages; improved sitemap/JSON-LD parsing and heuristics to favor specific assessment pages.

Latency & performance - Retrieval micro-benchmarks show ~4–6ms per query for BM25+FAISS on current catalog; LLM calls are external and variable.

Deployment - Dockerfile and entrypoint included. Optionally pre-warm embeddings for faster cold-start.

AI tools - Sentence-transformers for embeddings; FAISS for vector search; Groq LLM client for phrasing/rerank (optional).